

NutriTrack

AI-Powered Food & Nutrition Tracker

Final Project Documentation

Created for: Sai Phani Anirudh

Version: 3.0

Status: Deployed & Production-Ready

Date: June 2026

NutriTrack

AI-Powered Local Food & Nutrition Tracker Scan any meal photo to instantly detect multiple food items, estimate calories/macros, and track your daily nutrition goals with a fully private, local-first stack.

[!NutriTrack](#) [!Flask](#) [!PWA](#) [!Privacy](#)

Key Features

- **Smart Food Scanner** Take or upload a photo of your plate to detect multiple food items in a single scan.
- **100% Local AI** Uses llava-phi3 running locally via Ollama. None of your food photos are sent to third-party cloud APIs.
- **Indian Cuisine Specialist** Built-in offline fallback database covering Biryani, Dosa, Dal, Paratha, and 80+ other regional foods.
- **Comprehensive Macro Tracking** Calculates Calories, Protein, Carbs, Fat, Fiber, Sugar, Sodium, and Cholesterol.
- **30-Day Progress History** Daily calorie budget bars, weekly macro breakdowns, and consecutive streak tracking.
- **Secure Offline Auth** Multi-device register/login with passwords hashed using SHA-256 (WebCrypto API).
- **Progressive Web App (PWA)** Install directly onto your mobile home screen with full offline capability and service worker caching.
- **Responsive Mobile Design** Bottom navigation bar with dynamic active-tab sync, and a dashboard quick-access banner for mobile diet planning.
- **SigLIP Food-Only Rejection Guard** Built-in zero-shot classifier that automatically filters out non-food uploads (e.g. hands, body parts, or household items) before wasting LLM API cycles.

System Architecture

...
Browser / PWA (Port 5000) backend/App.py (Flask REST API Server) Auth & User Session logs Local SQLite Database (Port 5002) llm/Llm_server.py (AI Inference Hub) Primary: Ollama/Llava Cloud: Groq (Optional)
...

AI Inference Engines

| Engine | Latency | Accuracy | Cloud/Local | Required Config | |---|---|---|---|---| | **Ollama / llava-phi3** | ~1520s (CPU) / <2s (GPU) | ~85% (High) | 100% Local | None (Default) | | **Groq Cloud (Llama 3)** | ~23s | ~88% (High) | Cloud API | GROQ_API_KEY | | **Moondream2** | ~30s+ (CPU) | ~50% (Low) | 100% Local | HF_TOKEN (Fallback) |

Production Deployments

The live, public version of the project is distributed across the following cloud infrastructure:

1. **Frontend & REST API Backend (Render):** Flask-based web service serving the static frontend assets and managing the user databases and JWT sessions.
Live Web Application URL: <https://nutritrack-k96f.onrender.com/>
2. Runtime: Python (Gunicorn WSGI)
3. Configuration: Guided by [gunicorn.conf.py](#) to dynamically bind to \$PORT assigned by Render, preventing startup timeouts.
4. **AI Inference Server (Hugging Face Spaces):** Python service hosted on Hugging Face spaces executing zero-shot image validation and LLM extraction.
Public Space API URL: <https://energyvenom-nutritrack-llm.hf.space>
- 5.
- 6.

Directory Layout

nutritrack/ frontend/ Frontend UI (HTML, CSS, JS, manifest, PWA Service Worker)
index.html Style.css App.js Foods.js sw.js icons/ backend/ Flask REST API backend
App.py Database.py llm/ Local AI inference server Llm_server.py docker/ Container
configurations Dockerfile.api Dockerfile.llm requirements.txt docker-compose.yml
.env.example setup.bat One-click Windows setup setup.sh One-click macOS/Linux setup

Quick Start (Local Development)

Prerequisites

- Python 3.10 or higher
- [Ollama](#) (installed and running)

Windows Setup

Clone & Navigate: powershell git clone
<https://github.com/SaiPhaniAnirudh/nutritrack.git> cd nutritrack

Run One-Click Installer: powershell setup.bat *This script creates the virtual environment, installs dependencies, sets up .env, and pulls the llava-phi3 model.*

Start the Services:

4. **Terminal A (AI Server):** powershell .venv\Scripts\activate.bat python llm\Llm_server.py

Terminal B (Flask Backend): powershell .venv\Scripts\activate.bat python backend\App.py

Access the App: Open <http://localhost:5000> in your browser.

macOS / Linux Setup

Clone & Navigate: bash git clone <https://github.com/SaiPhaniAnirudh/nutritrack.git> cd nutritrack

Run Installer: bash chmod +x setup.sh ./setup.sh

Start the Services:

4. **Terminal A (AI Server):** bash source .venv/bin/activate python llm/Llm_server.py

Terminal B (Flask Backend): bash source .venv/bin/activate python backend/App.py

Access the App: Open <http://localhost:5000> in your browser.

Running with Docker

You can spin up the entire local ecosystem in one command.

```
```bash
```

# Start the stack (Ollama + Flask API + AI Server)

```
docker compose up -d
```

## Watch container logs

```
docker compose logs -f
```

## Stop the stack

```
docker compose down ``
```

**First Run Note:** Docker will automatically download the ~2.9 GB `llava-phi3` model inside the Ollama container. This may take a few minutes depending on your internet connection.

## NVIDIA GPU Support in Docker

To run inference at sub-2-second speeds, uncomment the `deploy resources` block inside [docker-compose.yml](#) under the `ollama` service to pass-through your CUDA GPU.

## Configuration (.env)

A default `.env` is automatically created during setup. You can customize settings by editing the file:

```
``env
```

## Flask JWT Signing Key (Keep secure)

```
JWT_SECRET_KEY=f99e7c0b4fb910106b690099840c1f9b17c6e8a175201b15a889ce793bf01324
```

## Local Ollama Configurations

```
OLLAMA_URL=http://localhost:11434 OLLAMA_MODEL=llava-phi3
```

## Optional: Groq Cloud API for ultra-fast (2s) cloud fallback inference

```
GROQ_API_KEY=gsk_...
```

## Optional: Persistent production database url (default is SQLite)

```
DATABASE_URL=postgresql://user:pass@loc
```

# alhost:5432/nutritrack

...

## API Endpoints

### 1. AI Analysis

- **POST /api/ai/analyze**
- **Payload:** { "image": "data:image/jpeg;base64,..." }
- **Response:** json { "description": "Chicken Curry and Steamed Rice", "items": [ { "food\_name": "Chicken Curry", "calories": 240, "protein\_g": 18.5, "carbs\_g": 8.0, "fat\_g": 15.0, "fiber\_g": 1.2, "sugar\_g": 2.0, "sodium\_mg": 680, "cholesterol\_mg": 75, "serving\_size": "1 bowl (200g)", "confidence": 90 }, { "food\_name": "Steamed Rice", "calories": 205, "protein\_g": 4.2, "carbs\_g": 44.5, "fat\_g": 0.4, "fiber\_g": 0.6, "sugar\_g": 0.1, "sodium\_mg": 5, "cholesterol\_mg": 0, "serving\_size": "1 cup (150g)", "confidence": 95 } ], "tips": "A balanced meal with good protein. Add some green vegetables for more fiber.", "\_meta": { "model": "Ollama/llava-phi3", "mode": "local\_llm", "latency\_ms": 15420 } }

### 2. User Authentication

| Method | Endpoint | Description | |---|---|---| | POST | /api/auth/register | Create a new user profile | | POST | /api/auth/login | Log in and receive a secure JWT | | POST | /api/auth/refresh | Refresh an expiring session token | | GET | /api/auth/me | Fetch profile configurations and calorie targets | | PUT | /api/auth/update | Update daily calorie/macro goals and profile |

### 3. Food Logs & Analytics

| Method | Endpoint | Description | |---|---|---| | GET | /api/logs | Fetch user entries (filters: ?date=YYYY-MM-DD or ?days=30) | | POST | /api/logs | Log a custom or scanned food item entry | | DELETE | /api/logs/<id> | Delete a logged food entry | | GET | /api/logs/summary | Summarize macro-nutrients over a window | | GET | /api/analytics/streak | Get the user's consecutive day logging streak |

## License

Distributed under the MIT License. See LICENSE for details.